!Notes

Order_details(degenerate dimension) will be created in the scope of the HW9 to be able to load data from 3NF layer(pre-fact table), not from source

Dim_dates were created within previous HW

–ROLE AND PRIVILEGES

```
GRANT USAGE ON SCHEMA bl_dm TO dwh_user;
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA bl_dm TO dwh_user;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA bl_dm TO dwh_user;
ALTER DEFAULT PRIVILEGES IN SCHEMA bl_dm
    GRANT SELECT, INSERT, UPDATE ON TABLES TO dwh_user;
ALTER DEFAULT PRIVILEGES IN SCHEMA bl_dm
    GRANT USAGE, SELECT ON SEQUENCES TO dwh_user;
```

-LOGGING FEATURE - reused from HW7

```
CREATE SEQUENCE IF NOT EXISTS bl_cl.seq_mta_load_logs_id START WITH 1 INCREMENT BY 1;

CREATE TABLE IF NOT EXISTS bl_cl.mta_load_logs (
    log_id         BIGINT DEFAULT nextval('bl_cl.seq_mta_load_logs_id') NOT NULL,
    log_dt         TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    procedure_name VARCHAR NOT NULL,
    log_status     VARCHAR NOT NULL,
    rows_affected  INT,
    log_message    TEXT,
    CONSTRAINT pk_mta_load_logs PRIMARY KEY (log_id),
    CONSTRAINT chk_mta_load_logs_status CHECK (log_status IN ('SUCCESS','ERROR'))
);

CREATE OR REPLACE PROCEDURE bl_cl.p_log(
    p_procedure_name VARCHAR,
    p_log_status     VARCHAR,
    p_rows_affected  INT,
    p_log_message    TEXT
)
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO bl_cl.mta_load_logs (procedure_name, log_status, rows_affected, log_message)
    VALUES (p_procedure_name, p_log_status, p_rows_affected, p_log_message);
END;
$$;

CREATE OR REPLACE FUNCTION bl_cl.get_load_summary()
RETURNS TABLE (
    log_id         BIGINT,
    log_dt         TIMESTAMP,
    procedure_name VARCHAR,
    log_status     VARCHAR,
    rows_affected  INT,
    log_message    TEXT
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT l.log_id, l.log_dt, l.procedure_name, l.log_status, l.rows_affected, l.log_message
    FROM bl_cl.mta_load_logs l
    ORDER BY l.log_id DESC;
END;
$$;

SELECT * FROM bl_cl.get_load_summary();
```

-DIM_PRODUCTS

The three-way join across ce_products > ce_subcategories > ce_categories denormalizes the 3NF hierarchy into one dimension row.A composite type t_product_row defines the shape of one product (product + its subcategory + its category), and the cursor loop fetches each conformed row into that single typed variable.

LOG:

log_id	log_dt	procedure_name	log_status	rows_affected	log_message
53	2026-07-21 20:11:18.232	load_dim_products	SUCCESS	0	Inserted: 0, Updated: 0 (composite type)
52	2026-07-21 20:10:55.002	load_dim_products	SUCCESS	5,000	Inserted: 5000, Updated: 0 (composite type)

TABLE DATA:

product_surr_id	product	product_subcategory_name	product_category_id	product_category_name	source_system	source_entity	product_src_id	insert_dt	update_dt
2,501	101	Appliances	103	Home & Kitchen	BL_3NF	CE_PRODUCTS	100	2026-07-21	2026-07-21
2,502	117	Laptop	104	Electronics	BL_3NF	CE_PRODUCTS	101	2026-07-21	2026-07-21
2,503	108	Snacks	101	Groceries	BL_3NF	CE_PRODUCTS	102	2026-07-21	2026-07-21
2,504	119	Non-Fiction	106	Books	BL_3NF	CE_PRODUCTS	103	2026-07-21	2026-07-21
2,505	102	Accessories	104	Electronics	BL_3NF	CE_PRODUCTS	104	2026-07-21	2026-07-21
2,506	111	Skincare	107	Beauty	BL_3NF	CE_PRODUCTS	105	2026-07-21	2026-07-21
2,507	113	Fresh Produce	101	Groceries	BL_3NF	CE_PRODUCTS	106	2026-07-21	2026-07-21
2,508	107	Beverages	101	Groceries	BL_3NF	CE_PRODUCTS	107	2026-07-21	2026-07-21
2,509	117	Laptop	104	Electronics	BL_3NF	CE_PRODUCTS	108	2026-07-21	2026-07-21
2,510	107	Beverages	101	Groceries	BL_3NF	CE_PRODUCTS	109	2026-07-21	2026-07-21
2,511	126	Men Clothing	102	Fashion	BL_3NF	CE_PRODUCTS	110	2026-07-21	2026-07-21
2,512	128	Shoes	102	Fashion	BL_3NF	CE_PRODUCTS	111	2026-07-21	2026-07-21
2,513	128	Shoes	102	Fashion	BL_3NF	CE_PRODUCTS	112	2026-07-21	2026-07-21
2,514	126	Men Clothing	102	Fashion	BL_3NF	CE_PRODUCTS	113	2026-07-21	2026-07-21

```
CREATE TYPE bl_cl.t_product_row AS (
    product_src_id           VARCHAR,
    product_subcategory_id   BIGINT,
    product_subcategory_name VARCHAR,
    product_category_id      BIGINT,
    product_category_name    VARCHAR
);
```

```
CREATE OR REPLACE PROCEDURE bl_cl.load_dim_products()
LANGUAGE plpgsql
AS $$
DECLARE
    v_rows_ins INT = 0;
    v_rows_upd INT = 0;
    v_tmp      INT = 0;
    v_row      bl_cl.t_product_row;
BEGIN
    FOR v_row IN
        SELECT
            p.product_id::VARCHAR       AS product_src_id,
            sub.product_subcategory_id,
            sub.product_subcategory_name,
            cat.product_category_id,
            cat.product_category_name
        FROM bl_3nf.ce_products p
        JOIN bl_3nf.ce_subcategories sub ON sub.product_subcategory_id = p.product_subcategory_id
        JOIN bl_3nf.ce_categories   cat ON cat.product_category_id     = sub.product_category_id
        WHERE p.product_id <> -1
    LOOP
        -- UPDATE if attributes changed
        UPDATE bl_dm.dim_products tgt
        SET product_subcategory_id   = v_row.product_subcategory_id,
            product_subcategory_name = v_row.product_subcategory_name,
            product_category_id      = v_row.product_category_id,
            product_category_name    = v_row.product_category_name,
            update_dt = CURRENT_DATE
        WHERE tgt.product_src_id = v_row.product_src_id
          AND tgt.source_system  = 'BL_3NF'
          AND (tgt.product_subcategory_name IS DISTINCT FROM v_row.product_subcategory_name
            OR tgt.product_category_name    IS DISTINCT FROM v_row.product_category_name);
        GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_upd = v_rows_upd + v_tmp;

        -- INSERT if new (one row per 3NF product)
        INSERT INTO bl_dm.dim_products (
            product_surr_id, product_subcategory_id, product_subcategory_name,
            product_category_id, product_category_name, source_system,
            source_entity, product_src_id, insert_dt, update_dt)
        SELECT nextval('bl_dm.seq_dim_products_surr_id'),
               v_row.product_subcategory_id, v_row.product_subcategory_name,
               v_row.product_category_id, v_row.product_category_name,
               'BL_3NF', 'CE_PRODUCTS', v_row.product_src_id,
               CURRENT_DATE, CURRENT_DATE
        WHERE NOT EXISTS (SELECT 1 FROM bl_dm.dim_products tgt
            WHERE tgt.product_src_id = v_row.product_src_id
              AND tgt.source_system  = 'BL_3NF');
        GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_ins = v_rows_ins + v_tmp;
    END LOOP;

    CALL bl_cl.p_log('load_dim_products', 'SUCCESS', v_rows_ins + v_rows_upd,
                     'Inserted: ' || v_rows_ins || ', Updated: ' || v_rows_upd || ' (composite type)');
EXCEPTION WHEN OTHERS THEN
    CALL bl_cl.p_log('load_dim_products', 'ERROR', NULL, SQLERRM);
END;
```

-DIM_CHANNELS

A declared cursor iterates the joined ce_subchannels >ce_channels result. DISTINCT ON (channel, subchannel) conforms the dimension: INT and US both supply the same channels, so the same channel/subchannel pair appears twice in 3NF and is collapsed to one DM row.

LOG:

log_id	log_dt	procedure_name	log_status	rows_affected	log_message
57	2026-07-21 22:55:24.722	load_dim_channels	SUCCESS	0	Inserted: 0, Updated: 0 (cursor FOR loop,
56	2026-07-21 22:55:23.480	load_dim_channels	SUCCESS	6	Inserted: 6, Updated: 0 (cursor FOR loop,

TABLE DATA:

subchannel_surr_id	subchannel	channel_id	channel	source_system	source_entity	subchannel_src_id	insert_dt	update_dt
13	BuyEverything	100	Marketplace	BL_3NF	CE_SUBCHANNELS	102	2026-07-21	2026-07-21
14	third-party business	100	Marketplace	BL_3NF	CE_SUBCHANNELS	105	2026-07-21	2026-07-21
15	Android App	102	Mobile App	BL_3NF	CE_SUBCHANNELS	103	2026-07-21	2026-07-21
16	iOS App	102	Mobile App	BL_3NF	CE_SUBCHANNELS	101	2026-07-21	2026-07-21
17	Desktop Web	104	Web	BL_3NF	CE_SUBCHANNELS	104	2026-07-21	2026-07-21
18	Mobile Web	104	Web	BL_3NF	CE_SUBCHANNELS	100	2026-07-21	2026-07-21

```
CREATE OR REPLACE PROCEDURE bl_cl.load_dim_channels()
LANGUAGE plpgsql
AS $$
DECLARE
    v_rows_ins INT = 0;
    v_rows_upd INT = 0;
    v_tmp      INT = 0;
    c_channels CURSOR FOR
        SELECT DISTINCT ON (ch.channel, sc.subchannel)
            sc.subchannel_id::VARCHAR AS subchannel_src_id,
            sc.subchannel,
            ch.channel_id,
            ch.channel
        FROM bl_3nf.ce_subchannels sc
        JOIN bl_3nf.ce_channels ch ON ch.channel_id = sc.channel_id
        WHERE sc.subchannel_id <> -1
        ORDER BY ch.channel, sc.subchannel, sc.subchannel_id;
BEGIN
    FOR rec IN c_channels LOOP
        UPDATE bl_dm.dim_channels tgt
        SET channel_id = rec.channel_id,
            update_dt  = CURRENT_DATE
        WHERE tgt.subchannel = rec.subchannel
          AND tgt.channel    = rec.channel
          AND tgt.source_system = 'BL_3NF'
          AND tgt.channel_id IS DISTINCT FROM rec.channel_id;
        GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_upd = v_rows_upd + v_tmp;

        INSERT INTO bl_dm.dim_channels (
            subchannel_surr_id, subchannel, channel_id, channel,
            source_system, source_entity, subchannel_src_id, insert_dt, update_dt)
        SELECT nextval('bl_dm.seq_dim_channels_surr_id'),
               rec.subchannel, rec.channel_id, rec.channel,
               'BL_3NF', 'CE_SUBCHANNELS', rec.subchannel_src_id,
               CURRENT_DATE, CURRENT_DATE
        WHERE NOT EXISTS (SELECT 1 FROM bl_dm.dim_channels tgt
            WHERE tgt.subchannel = rec.subchannel
              AND tgt.channel    = rec.channel
              AND tgt.source_system = 'BL_3NF');
        GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_ins = v_rows_ins + v_tmp;
    END LOOP;

    CALL bl_cl.p_log('load_dim_channels', 'SUCCESS', v_rows_ins + v_rows_upd,
                     'Inserted: ' || v_rows_ins || ', Updated: ' || v_rows_upd || ' (cursor FOR loop, conformed)');
EXCEPTION WHEN OTHERS THEN
    CALL bl_cl.p_log('load_dim_channels', 'ERROR', NULL, SQLERRM);
END;
$$;
```

-CE_EMPLOYEES

An explicit refcursor is opened on the source query, then rows are pulled one at a time with FETCH ... INTO until NOT FOUND, and the cursor is closed manually. This demonstrates manual cursor control in contrast to the automatic cursor FOR loop used for channels. Sales reps are non-conforming (no employee appears in both sources), so each 3NF row becomes one dimension row.

LOG:

log_id	log_dt	procedure_name	log_status	rows_affected	log_message
62	2026-07-21 23:17:16.235	load_dim_employees	SUCCESS	0	Inserted: 0, Updated: 0 (cursor variable)
61	2026-07-21 23:17:15.171	load_dim_employees	SUCCESS	200	Inserted: 200, Updated: 0 (cursor variable)

TABLE DATA:

sales_rep_surr_id	sales_rep_first_name	sales_rep_last_name	sales_rep_email	source_system	source_entity	sales_rep_src_id	insert_dt	update_dt
-1	n. a.	n. a.	n. a.	MANUAL	MANUAL	n. a.	1900-01-01	1900-01-01
1	Michael	Turner	michael.turner@novamart.c	BL_3NF	CE_EMPLOYEES	100	2026-07-21	2026-07-21
2	Joshua	Turner	joshua.turner@novamart.co	BL_3NF	CE_EMPLOYEES	101	2026-07-21	2026-07-21
3	Lauren	Stewart	lauren.stewart@novamart.c	BL_3NF	CE_EMPLOYEES	102	2026-07-21	2026-07-21
4	Gregory	Johnson	gregory.johnson@novamart	BL_3NF	CE_EMPLOYEES	103	2026-07-21	2026-07-21
5	Olivia	White	olivia.white@novamart.com	BL_3NF	CE_EMPLOYEES	104	2026-07-21	2026-07-21
6	Gary	Rodriguez	gary.rodriguez@novamart.c	BL_3NF	CE_EMPLOYEES	105	2026-07-21	2026-07-21
7	Rachel	Flores	rachel.flores@novamart.cor	BL_3NF	CE_EMPLOYEES	106	2026-07-21	2026-07-21

```
CREATE OR REPLACE PROCEDURE bl_cl.load_dim_employees()
LANGUAGE plpgsql
AS $$
DECLARE
    v_rows_ins INT = 0;
    v_rows_upd INT = 0;
    v_tmp      INT = 0;
    v_cur      refcursor;
    v_src_id   VARCHAR;
    v_fname    VARCHAR;
    v_lname    VARCHAR;
    v_email    VARCHAR;
BEGIN
    OPEN v_cur FOR
        SELECT sales_rep_id::VARCHAR AS sales_rep_src_id,
               sales_rep_first_name, sales_rep_last_name, sales_rep_email
        FROM bl_3nf.ce_employees
        WHERE sales_rep_id <> -1;

    LOOP
        FETCH v_cur INTO v_src_id, v_fname, v_lname, v_email;
        EXIT WHEN NOT FOUND;

        UPDATE bl_dm.dim_employees tgt
        SET sales_rep_first_name = v_fname,
            sales_rep_last_name  = v_lname,
            sales_rep_email      = v_email,
            update_dt = CURRENT_DATE
        WHERE tgt.sales_rep_src_id = v_src_id
          AND tgt.source_system    = 'BL_3NF'
          AND (tgt.sales_rep_first_name IS DISTINCT FROM v_fname
            OR tgt.sales_rep_last_name  IS DISTINCT FROM v_lname
            OR tgt.sales_rep_email      IS DISTINCT FROM v_email);
        GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_upd = v_rows_upd + v_tmp;

        INSERT INTO bl_dm.dim_employees (
            sales_rep_surr_id, sales_rep_first_name, sales_rep_last_name,
            sales_rep_email, source_system, source_entity, sales_rep_src_id,
            insert_dt, update_dt)
        SELECT nextval('bl_dm.seq_dim_employees_surr_id'),
               v_fname, v_lname, v_email, 'BL_3NF', 'CE_EMPLOYEES', v_src_id,
               CURRENT_DATE, CURRENT_DATE
        WHERE NOT EXISTS (SELECT 1 FROM bl_dm.dim_employees tgt
            WHERE tgt.sales_rep_src_id = v_src_id AND tgt.source_system = 'BL_3NF');
        GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_ins = v_rows_ins + v_tmp;
    END LOOP;
    CLOSE v_cur;   -- release the cursor
    CALL bl_cl.p_log('load_dim_employees', 'SUCCESS', v_rows_ins + v_rows_upd,
                     'Inserted: ' || v_rows_ins || ', Updated: ' || v_rows_upd || ' (cursor variable)');
EXCEPTION WHEN OTHERS THEN
    CALL bl_cl.p_log('load_dim_employees', 'ERROR', NULL, SQLERRM);
END;
$$;
```

-DIM_LOCATIONS UPSERT (INSERT ... ON CONFLICT)

A single set-based INSERT ... ON CONFLICT ... DO UPDATE loads the ce_cities > ce_states > ce_countries in one statement. The WHERE clause on the DO UPDATE means unchanged rows are skipped entirely, so a re-run reports zero rows affected.

LOG:

log_id	log_dt	procedure_name	log_status	rows_affected	log_message
71	2026-07-22 22:36:20.486	load_dim_locations	SUCCESS	0	Rows affected: 0 (upsert ON CONFLICT)
70	2026-07-22 22:36:19.533	load_dim_locations	SUCCESS	0	Rows affected: 0 (upsert ON CONFLICT)
69	2026-07-22 22:29:19.092	load_dim_locations	SUCCESS	0	Rows affected: 0 (upsert ON CONFLICT)
68	2026-07-21 23:21:22.426	load_dim_locations	SUCCESS	0	Rows affected: 0 (upsert ON CONFLICT)
67	2026-07-21 23:21:21.635	load_dim_locations	SUCCESS	194	Rows affected: 194 (upsert ON CONFLICT)

TABLE DATA:

city_surr_id	city_name	state_id	state	country_id	country_name	source_system	source_entity	city_name_src_id	insert_dt	update_dt
-1	n. a.	-1	n. a.	-1	n. a.	MANUAL	MANUAL	-1	1900-01-01	1900-01-01
1	Hyderabad	239	India_state	106	India	BL_3NF	CE_CITIES	788	2026-07-21	2026-07-21
2	Calgary	242	Canada_state	104	Canada	BL_3NF	CE_CITIES	789	2026-07-21	2026-07-21
3	Berlin	241	Germany_state	102	Germany	BL_3NF	CE_CITIES	790	2026-07-21	2026-07-21
4	Melbourne	244	Australia_state	100	Australia	BL_3NF	CE_CITIES	791	2026-07-21	2026-07-21
5	Cologne	241	Germany_state	102	Germany	BL_3NF	CE_CITIES	792	2026-07-21	2026-07-21
6	Perth	244	Australia_state	100	Australia	BL_3NF	CE_CITIES	793	2026-07-21	2026-07-21
7	Vancouver	242	Canada_state	104	Canada	BL_3NF	CE_CITIES	794	2026-07-21	2026-07-21
8	Abu Dhabi	240	UAE_state	101	UAE	BL_3NF	CE_CITIES	795	2026-07-21	2026-07-21
9	Lyon	236	France_state	103	France	BL_3NF	CE_CITIES	796	2026-07-21	2026-07-21
10	Riyadh	238	Saudi Arabia_stat	105	Saudi Arabia	BL_3NF	CE_CITIES	797	2026-07-21	2026-07-21
11	Frankfurt	241	Germany_state	102	Germany	BL_3NF	CE_CITIES	798	2026-07-21	2026-07-21
12	Karachi	243	Pakistan_state	108	Pakistan	BL_3NF	CE_CITIES	799	2026-07-21	2026-07-21
13	Sharjah	240	UAE_state	101	UAE	BL_3NF	CE_CITIES	800	2026-07-21	2026-07-21
14	Lahore	243	Pakistan_state	108	Pakistan	BL_3NF	CE_CITIES	801	2026-07-21	2026-07-21

```
ALTER TABLE bl_dm.dim_locations
ADD CONSTRAINT uq_dim_locations_bk UNIQUE (city_name_src_id, source_system);

CREATE OR REPLACE PROCEDURE bl_cl.load_dim_locations()
LANGUAGE plpgsql
AS $$
DECLARE
    v_rows_aff INT = 0;
BEGIN
    INSERT INTO bl_dm.dim_locations (
        city_surr_id, city_name, state_id, state, country_id, country_name,
        source_system, source_entity, city_name_src_id, insert_dt, update_dt)
    SELECT
        nextval('bl_dm.seq_dim_locations_surr_id'),
        ci.city_name, st.state_id, st.state_name, cn.country_id, cn.country_name,
        'BL_3NF', 'CE_CITIES', ci.city_id::VARCHAR,
        CURRENT_DATE, CURRENT_DATE
    FROM bl_3nf.ce_cities ci
    JOIN bl_3nf.ce_states    st ON st.state_id   = ci.state_id
    JOIN bl_3nf.ce_countries cn ON cn.country_id = st.country_id
    WHERE ci.city_id <> -1
    ON CONFLICT (city_name_src_id, source_system) DO UPDATE
        SET city_name    = EXCLUDED.city_name,
            state_id     = EXCLUDED.state_id,
            state        = EXCLUDED.state,
            country_id   = EXCLUDED.country_id,
            country_name = EXCLUDED.country_name,
            update_dt    = CURRENT_DATE
        WHERE dim_locations.city_name    IS DISTINCT FROM EXCLUDED.city_name
           OR dim_locations.state        IS DISTINCT FROM EXCLUDED.state
           OR dim_locations.country_name IS DISTINCT FROM EXCLUDED.country_name;
    GET DIAGNOSTICS v_rows_aff = ROW_COUNT;

    CALL bl_cl.p_log('load_dim_locations', 'SUCCESS', v_rows_aff,
                     'Rows affected: ' || v_rows_aff || ' (upsert ON CONFLICT)');
EXCEPTION WHEN OTHERS THEN
    CALL bl_cl.p_log('load_dim_locations', 'ERROR', NULL, SQLERRM);
END;
$$;
```

-CE_CUSTOMERS_SCD

Changed customers have their active version expired (is_active='N', end_dt set to yesterday) and a new active version inserted with an open end date of 9999-12-31. Because the 3NF layer already performs SCD2 versioning, the DM mirrors those versions one-to-one rather than re-deriving history. This keeps the version chain intact for the fact table, which will resolve each order to the correct customer version.

```
CREATE OR REPLACE PROCEDURE bl_cl.load_dim_customers_scd()
LANGUAGE plpgsql
AS $$
DECLARE
    v_rows_ins INT = 0;
    v_rows_exp INT = 0;
    v_tmp      INT = 0;
BEGIN
    -- Expireversions whose attributes changed vs the 3NF row
    UPDATE bl_dm.dim_customers_scd tgt
    SET is_active = 'N',
        end_dt    = CURRENT_DATE - 1,
        update_dt = CURRENT_DATE
    FROM (SELECT customer_id::VARCHAR AS customer_src_id,
                 customer_first_name, customer_last_name, customer_email,
                 customer_age, customer_gender, customer_segment
          FROM bl_3nf.ce_customers_scd
          WHERE is_active = 'Y' AND customer_id <> -1) src
    WHERE tgt.customer_src_id = src.customer_src_id
      AND tgt.source_system   = 'BL_3NF'
      AND tgt.is_active = 'Y'
      AND (tgt.customer_first_name IS DISTINCT FROM src.customer_first_name
        OR tgt.customer_last_name  IS DISTINCT FROM src.customer_last_name
        OR tgt.customer_email      IS DISTINCT FROM src.customer_email
        OR tgt.customer_age        IS DISTINCT FROM src.customer_age
        OR tgt.customer_gender     IS DISTINCT FROM src.customer_gender
        OR tgt.customer_segment    IS DISTINCT FROM src.customer_segment);
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_exp = v_rows_exp + v_tmp;

    -- Insert new active version
    INSERT INTO bl_dm.dim_customers_scd (
        customer_surr_id, customer_first_name, customer_last_name, customer_email,
        customer_age, customer_gender, customer_segment, start_dt, end_dt, is_active,
        source_system, source_entity, customer_src_id, insert_dt, update_dt)
    SELECT nextval('bl_dm.seq_dim_customers_surr_id'),
           src.customer_first_name, src.customer_last_name, src.customer_email,
           src.customer_age, src.customer_gender, src.customer_segment,
           CURRENT_DATE, DATE '9999-12-31', 'Y',
           'BL_3NF', 'CE_CUSTOMERS_SCD', src.customer_src_id,
           CURRENT_DATE, CURRENT_DATE
    FROM (SELECT customer_id::VARCHAR AS customer_src_id,
                 customer_first_name, customer_last_name, customer_email,
                 customer_age, customer_gender, customer_segment
          FROM bl_3nf.ce_customers_scd
          WHERE is_active = 'Y' AND customer_id <> -1) src
    WHERE NOT EXISTS (
        SELECT 1 FROM bl_dm.dim_customers_scd tgt
        WHERE tgt.customer_src_id = src.customer_src_id
          AND tgt.source_system   = 'BL_3NF'
          AND tgt.is_active = 'Y');
    GET DIAGNOSTICS v_tmp = ROW_COUNT; v_rows_ins = v_rows_ins + v_tmp;

    CALL bl_cl.p_log('load_dim_customers_scd', 'SUCCESS', v_rows_ins + v_rows_exp,
                     'Inserted: ' || v_rows_ins || ', Expired: ' || v_rows_exp || ' (SCD2)');
EXCEPTION WHEN OTHERS THEN
    CALL bl_cl.p_log('load_dim_customers_scd', 'ERROR', NULL, SQLERRM);
END;
$$;
```

LOG:

log_id	log_dt	procedure_name	log_status	rows_affected	log_message
73	2026-07-22 22:37:00.775	load_dim_customers_scd	SUCCESS	0	Inserted: 0, Expired: 0 (SCD2)
72	2026-07-22 22:36:54.618	load_dim_customers_scd	SUCCESS	289,206	Inserted: 289206, Expired: 0 (SCD2)

TABLE DATA:

customer_surr_id	customer_first_name	customer_last_name	customer_email	cus	cust	cu	start_dt	end_dt	is_active	source_system	source_entity	custo	insert_dt	update_
1	Matthew	Sanchez	matthew.sanchez000001@gmail.com	n. a.	n. a.	n. a.	2026-07-22	9999-12-31	Y	BL_3NF	CE_CUSTOMERS_SCD	100	2026-07-22	2026-07-22
901	Lina	Al-Naqbi	lina.al-naqbi000935@gmail.com	n. a.	n. a.	n. a.	2026-07-22	9999-12-31	Y	BL_3NF	CE_CUSTOMERS_SCD	1000	2026-07-22	2026-07-22
9,901	Rajesh	Agarwal	rajesh.agarwal010257@gmail.com	n. a.	n. a.	n. a.	2026-07-22	9999-12-31	Y	BL_3NF	CE_CUSTOMERS_SCD	10000	2026-07-22	2026-07-22
99,901	Ravi	Mukherjee	ravi.mukherjee103643@outlook.com	n. a.	n. a.	n. a.	2026-07-22	9999-12-31	Y	BL_3NF	CE_CUSTOMERS_SCD	100000	2026-07-22	2026-07-22
99,902	Marcel	Leroy	marcel.leroy103644@yahoo.com	n. a.	n. a.	n. a.	2026-07-22	9999-12-31	Y	BL_3NF	CE_CUSTOMERS_SCD	100001	2026-07-22	2026-07-22
99,903	Fatima	Al-Shehhi	fatima.al-shehhi103645@icloud.com	n. a.	n. a.	n. a.	2026-07-22	9999-12-31	Y	BL_3NF	CE_CUSTOMERS_SCD	100002	2026-07-22	2026-07-22
99,904	Susan	Gomez	susan.gomez103646@gmail.com	n. a.	n. a.	n. a.	2026-07-22	9999-12-31	Y	BL_3NF	CE_CUSTOMERS_SCD	100003	2026-07-22	2026-07-22
99,905	Maryam	Akhtar	maryam.akhtar103647@yahoo.com	n. a.	n. a.	n. a.	2026-07-22	9999-12-31	Y	BL_3NF	CE_CUSTOMERS_SCD	100004	2026-07-22	2026-07-22
99,906	Samantha	Young	samantha.young103648@gmail.com	n. a.	n. a.	n. a.	2026-07-22	9999-12-31	Y	BL_3NF	CE_CUSTOMERS_SCD	100005	2026-07-22	2026-07-22
99,907	Karim	Al-Shehhi	karim.al-shehhi103649@yahoo.com	n. a.	n. a.	n. a.	2026-07-22	9999-12-31	Y	BL_3NF	CE_CUSTOMERS_SCD	100006	2026-07-22	2026-07-22
99,908	Nicole	Hernandez	nicole.hernandez103650@gmail.com	n. a.	n. a.	n. a.	2026-07-22	9999-12-31	Y	BL_3NF	CE_CUSTOMERS_SCD	100007	2026-07-22	2026-07-22
99,909	Janet	Diaz	janet.diaz103651@yahoo.com	n. a.	n. a.	n. a.	2026-07-22	9999-12-31	Y	BL_3NF	CE_CUSTOMERS_SCD	100008	2026-07-22	2026-07-22
99,910	Anas	Al-Balushi	anas.al-balushi103652@gmail.com	n. a.	n. a.	n. a.	2026-07-22	9999-12-31	Y	BL_3NF	CE_CUSTOMERS_SCD	100009	2026-07-22	2026-07-22
9,902	Hala	Al-Amri	hala.al-amri010258@icloud.com	n. a.	n. a.	n. a.	2026-07-22	9999-12-31	Y	BL_3NF	CE_CUSTOMERS_SCD	10001	2026-07-22	2026-07-22
99,911	Jacob	Jackson	jacob.jackson103653@gmail.com	n. a.	n. a.	n. a.	2026-07-22	9999-12-31	Y	BL_3NF	CE_CUSTOMERS_SCD	100010	2026-07-22	2026-07-22

Data change SCD2 screenshots:

Source:

C2 FabriceUPD

SRC_INTERNATIONAL_SALES.csv

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	ORDER_ID	CUSTOME	CUSTOMER_FIRST_NAME	CUSTOME	CUSTOME	ORDER_D	COUNTRY	CITY_INT	CUSTOME	PRODUCT_	CATEGORY	SUBCATEC	PRICE	QUANTITY DI
2	OINT0000	CINT0475	FabriceUPD	Simon	fabrice.sin	########	France	Toulouse	Premium	PROD0070	Groceries	Dairy	35.61	3
3	OINT0000	CINT0452	Shazia	Chaudhry	shazia.cha	########	Pakistan	Karachi	New	PROD0000	Fashion	Shoes	37.64	1
4	OINT0000	CINT0259	Ronald	Garcia	ronald.gar	########	United Kin	Manchest	New	PROD0001	Fashion	Shoes	63.56	3
5	OINT0000	CINT0406	Arun	Reddy	arun.reddy	########	India	Delhi	Returning	PROD0040	Fashion	Shoes	47.37	1
6	OINT0000	CINT0177	Martine	Dupont	martine.du	########	France	Marseille	Returning	PROD0014	Fashion	Women Cl	28.68	6
7	OINT0000	CINT0983	Anja	Koch	anja.koch(	########	Germany	Berlin	Premium	PROD0004	Fashion	Bags	246.51	1
8	OINT0000	CINT1080	Najla	Al-Hashim	najla.al-ha	########	UAE	Ajman	New	PROD0071	Groceries	Snacks	11.5	6
9	OINT0000	CINT1409	Samuel	Miller	samuel.mi	########	Canada	Montreal	Premium	PROD0035	Fashion	Bags	244.65	1
10	OINT0000	CINT0358	Sunita	Reddy	sunita.red	########	India	Hyderabac	Loyal	PROD011(	Electronic	Headphon	1786.41	1
11	OINT0000	CINT1350	Salma	Al-Shamsi	salma.al-s	########	UAE	Ajman	Returning	PROD0174	Beauty	Skincare	114.66	2
12	OINT0000	CINT0081	Carol	Evans	carol.evan	########	Canada	Montreal	Loyal	PROD0098	Electronic	Accessorie	1199.98	7
13	OINT0000	CINT0358	Benjamin	Torres	benjamin.t	########	United Kin	Liverpool	New	PROD0143	Home & Ki	Appliances	192.98	7
14	OINT0000	CINT1417	Michelle	Jackson	michelle.ja	########	United Kin	Manchest	Returning	PROD0207	Sports	Sportswea	209.58	4

SCD2 table:

```
SELECT * FROM bl_dm.dim_customers_scd WHERE customer_src_id = '578511';
```

omers_scd 1 | dim_customers_scd 2 | dim_customers_scd 3

* FROM bl_dm.dim_customers_scd WHERE customer_src_id = '57 — Enter a SQL expression to filter results (use Ctrl+Space)

cus	customer_first_name	cu	customer_email	cust	c	start	end_c	source_system	source_entity	customer_src_ic	insert_dt	update_dt
578,412	FabriceUPD	Simon	fabrice.simon047528@hotmail.com	n. a.	n. a.	2026-07-23	9999-12-31	BL_3NF	CE_CUSTOMERS_SCD	578511	2026-07-23	2026-07-23